# The University of Azad Jammu and Kashmir Muzaffarabad




**Jawahir Ali**

**2024-SE-34**

**Digital Image Processing _Lab**

**5th Semester**

**2024-28 (SE)**

**Lab_02**

## Department Of Software Engineering

## Step 1: Create the Project and Virtual Environment

Creating a project folder named dip-lab-02 and a virtual environment (.venv) so the project's libraries stay separate from the system Python. After activating it, I installed NumPy, OpenCV, and Matplotlib, and verified that the imports work and that the Python executable points inside .venv. I also added .venv/ to .gitignore so it is not submitted.

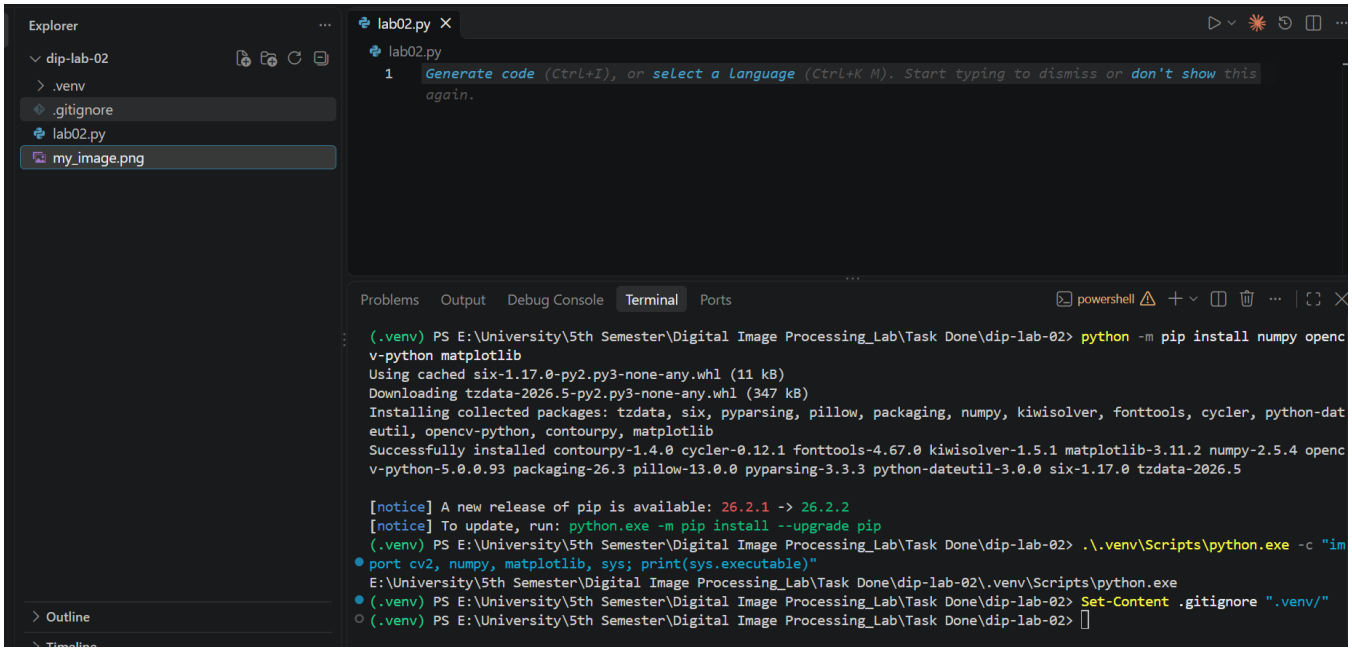



*Figure 1*

## Step 2: Load the Image

I loaded my selected color image using cv.imread(). The code raises a FileNotFoundError if the image is missing or unreadable. The output shows the image shape (height, width, channels) and its data type (uint8).

```python
# Step 2: Load the image
bgr = cv.imread(IMAGE_PATH)
if bgr is None:
    raise FileNotFoundError(f"Image not found or unreadable: {IMAGE_PATH}")

print("Step 2: Load")
print("Image shape (H, W, C):", bgr.shape)
print("Data type:", bgr.dtype)
```

*Figure 2*

## Output:

```
(.venv) PS E:\University\5th Semester\Digital Image Processing_Lab\Task Done\dip-lab-02> & "e:\University
\5th Semester\Digital Image Processing_Lab\Task Done\dip-lab-02\.venv\Scripts\python.exe" "e:/University/
5th Semester/Digital Image Processing_Lab/Task Done/dip-lab-02/lab02.py"
Step 2: Load
Image shape (H, W, C): (506, 528, 3)
Data type: uint8
```

*Figure 3*

## Step 3: Inspect the Image

I printed the image height, width, and number of channels. I also printed the top-left and center pixel values using [row, column] indexing. Finally, I printed the global minimum/maximum and the per-channel minimum/maximum values. OpenCV stores channels in BGR order.

```python
# Step 3: Inspect
h, w, c = bgr.shape
cy, cx = h // 2, w // 2

print("\n=== Step 3: Inspect ===")
print("Height:", h)
print("Width:", w)
print("Channels:", c)
print("Top-left pixel [0, 0] (BGR):", bgr[0, 0])
print(f"Center pixel [{cy}, {cx}] (BGR):", bgr[cy, cx])
print("Global min:", bgr.min(), "| Global max:", bgr.max())
for i, name in enumerate(["Blue", "Green", "Red"]):  # OpenCV order is BGR
    print(f"{name} channel -> min: {bgr[:, :, i].min()}, max: {bgr[:, :, i].max()}")
```

*Figure 4*

## Output:

```
=== Step 3: Inspect ===
Height: 506
Width: 528
Channels: 3
Top-left pixel [0, 0] (BGR): [29 29 29]
Center pixel [253, 264] (BGR): [97 55  6]
Global min: 0 | Global max: 255
Blue channel -> min: 0, max: 255
Green channel -> min: 0, max: 255
Red channel -> min: 0, max: 255
```

*Figure 5*

## Step 4: Convert Colors and Separate Channels

Converting the image from BGR to RGB so Matplotlib shows the correct colors, and created a grayscale version. Then separated the red, green, and blue channels and displayed everything in a 2×3 figure: RGB, red, green, blue, grayscale, and the unconverted BGR image. The unconverted BGR image looks wrong because Matplotlib expects RGB order. I also converted the center pixel's RGB value to a HEX color code.

```
# Step 4: Colors and channels
rgb = cv.cvtColor(bgr, cv.COLOR_BGR2RGB)
gray = cv.cvtColor(bgr, cv.COLOR_BGR2GRAY)
red, green, blue = rgb[:, :, 0], rgb[:, :, 1], rgb[:, :, 2]

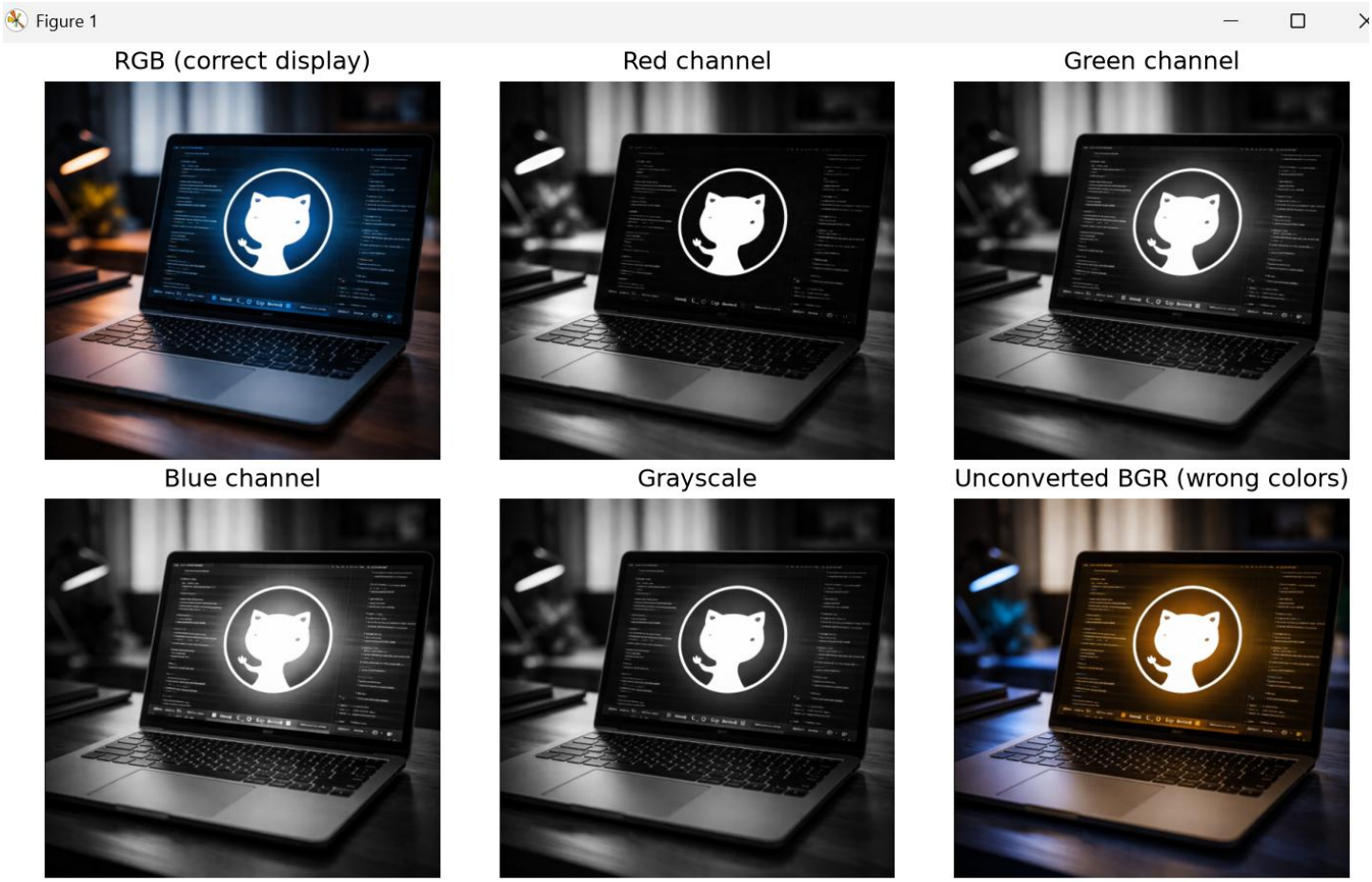
fig, axes = plt.subplots(2, 3, figsize=(14, 8))
panels = [
    (rgb, "RGB (correct display)", None),
    (red, "Red channel", "gray"),
    (green, "Green channel", "gray"),
    (blue, "Blue channel", "gray"),
    (gray, "Grayscale", "gray"),
    (bgr, "Unconverted BGR (wrong colors)", None),
]
for ax, (img, title, cmap) in zip(axes.ravel(), panels):
    ax.imshow(img, cmap=cmap)
    ax.set_title(title)
    ax.axis("off")
plt.tight_layout()
plt.savefig("channels_figure.png", dpi=120)

r, g, b = (int(v) for v in rgb[cy, cx])
hex_color = "#{:02X}{:02X}{:02X}".format(r, g, b)
print("\nStep 4: Colors")
print(f"Center pixel RGB: ({r}, {g}, {b})")
print("Center pixel HEX:", hex_color)
```

## Output:

```
Step 4: Colors
Center pixel RGB: (6, 55, 97)
Center pixel HEX: #063761
(.venv) PS E:\University\5th Semester\Digital Image Processing_Lab\Task Done\dip-la
```

## plt.show()




## Step 5: Crop, Resize, and Save

Cropping an exact 100×100 region from the center using NumPy slicing image[y_start:y_end, x_start:x_end] and confirmed its shape. I resized the crop to 200×200 with cv.resize() and displayed the original, crop, and resized image side by side. I converted the resized image back to BGR and saved it as resized_crop.png with cv.imwrite(), which returned True to confirm it saved.

```python
#  Step 5: Crop, resize, save
y_start, x_start = max(cy - 50, 0), max(cx - 50, 0)
y_end, x_end = y_start + 100, x_start + 100
crop = rgb[y_start:y_end, x_start:x_end]
print("\nStep 5: Crop / Resize")
print("Crop shape:", crop.shape)

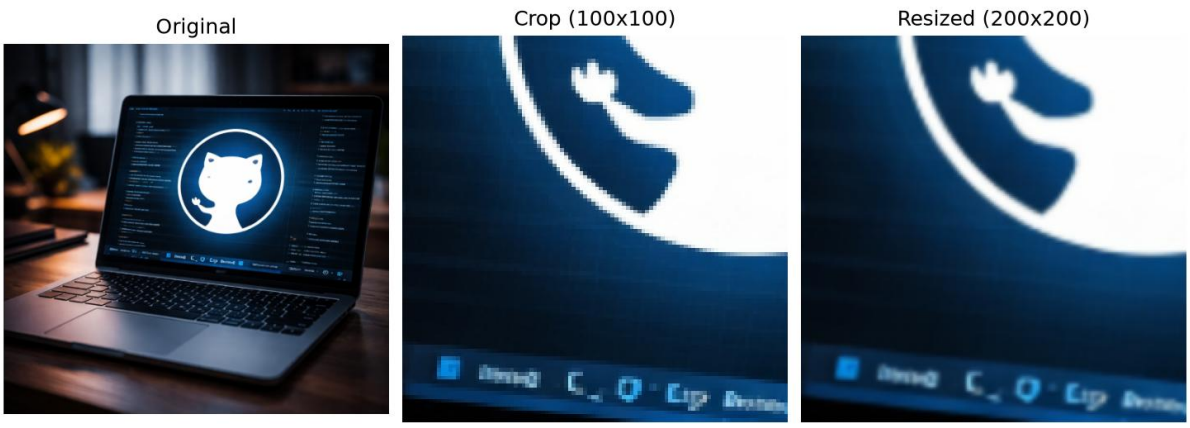
resized = cv.resize(crop, (200, 200))  # (width, height)
print("Resized shape:", resized.shape)

fig, axes = plt.subplots(1, 3, figsize=(14, 5))
for ax, img, title in zip(
    axes,
    [rgb, crop, resized],
    ["Original", "Crop (100x100)", "Resized (200x200)"],
):
    ax.imshow(img)
    ax.set_title(title)
    ax.axis("off")
plt.tight_layout()
plt.savefig("crop_resize_figure.png", dpi=120)

resized_bgr = cv.cvtColor(resized, cv.COLOR_RGB2BGR)
ok = cv.imwrite("resized_crop.png", resized_bgr)
print("cv.imwrite saved resized_crop.png successfully:", ok)
```

## Output

```
Step 5: Crop / Resize
Crop shape: (100, 100, 3)
Resized shape: (200, 200, 3)
cv.imwrite saved resized_crop.png successfully: True
```



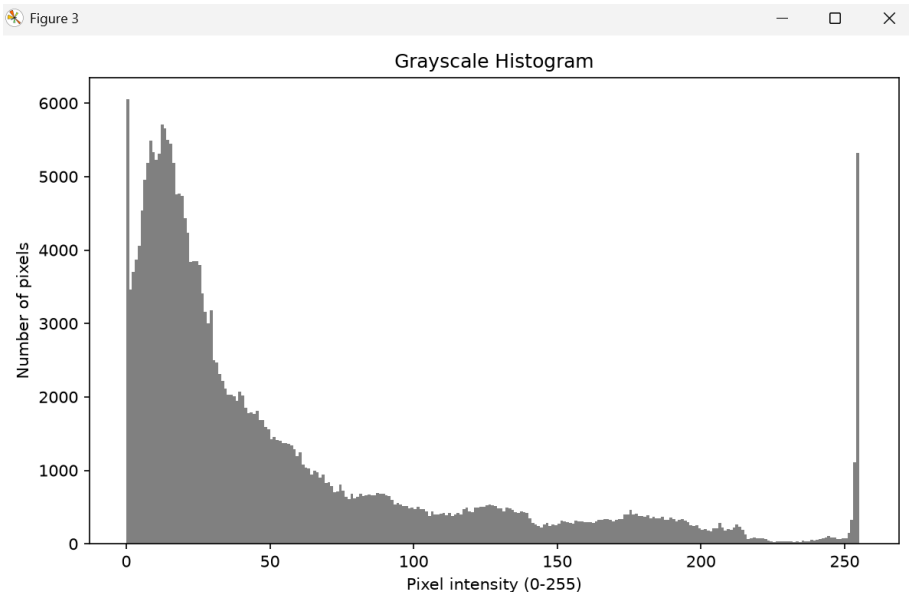

## Step 6: Histogram

I flattened the grayscale image and plotted a 256-bin histogram over the range 0–256. It shows how many pixels exist at each brightness level from 0 (black) to 255 (white). The plot is saved as grayscale_histogram.png.

```
#  Step 6: Histogram
plt.figure(figsize=(8, 5))
plt.hist(gray.flatten(), bins=256, range=(0, 256), color="gray")
plt.title("Grayscale Histogram")
plt.xlabel("Pixel intensity (0-255)")
plt.ylabel("Number of pixels")
plt.tight_layout()
plt.savefig("grayscale_histogram.png", dpi=120)
print("\nStep 6: Histogram")
print("Mean gray intensity:", round(float(gray.mean()), 2))
```

## Output:

```
Step 6: Histogram
Mean gray intensity: 54.94
```




## Step 7: Bit Depth

Creating an 8-bit horizontal gradient from 0 to 255 and quantized it to 1-bit, 2-bit, and 4-bit. These have 2, 4, and 16 intensity levels, respectively. The figure shows that fewer bits produce visible banding. The unique-value counts (256, 2, 4, 16) confirm the quantization worked.

```python
# Step 7: Bit depth
gradient = np.tile(np.arange(256, dtype=np.uint8), (100, 1))


def quantize(img, bits):
    levels = 2 ** bits
    step = 256 // levels
    scale = 255 // (levels - 1)
    return ((img // step) * scale).astype(np.uint8)


grads = [
    (gradient, "8-bit (256 levels)"),
    (quantize(gradient, 1), "1-bit (2 levels)"),
    (quantize(gradient, 2), "2-bit (4 levels)"),
    (quantize(gradient, 4), "4-bit (16 levels)"),
]

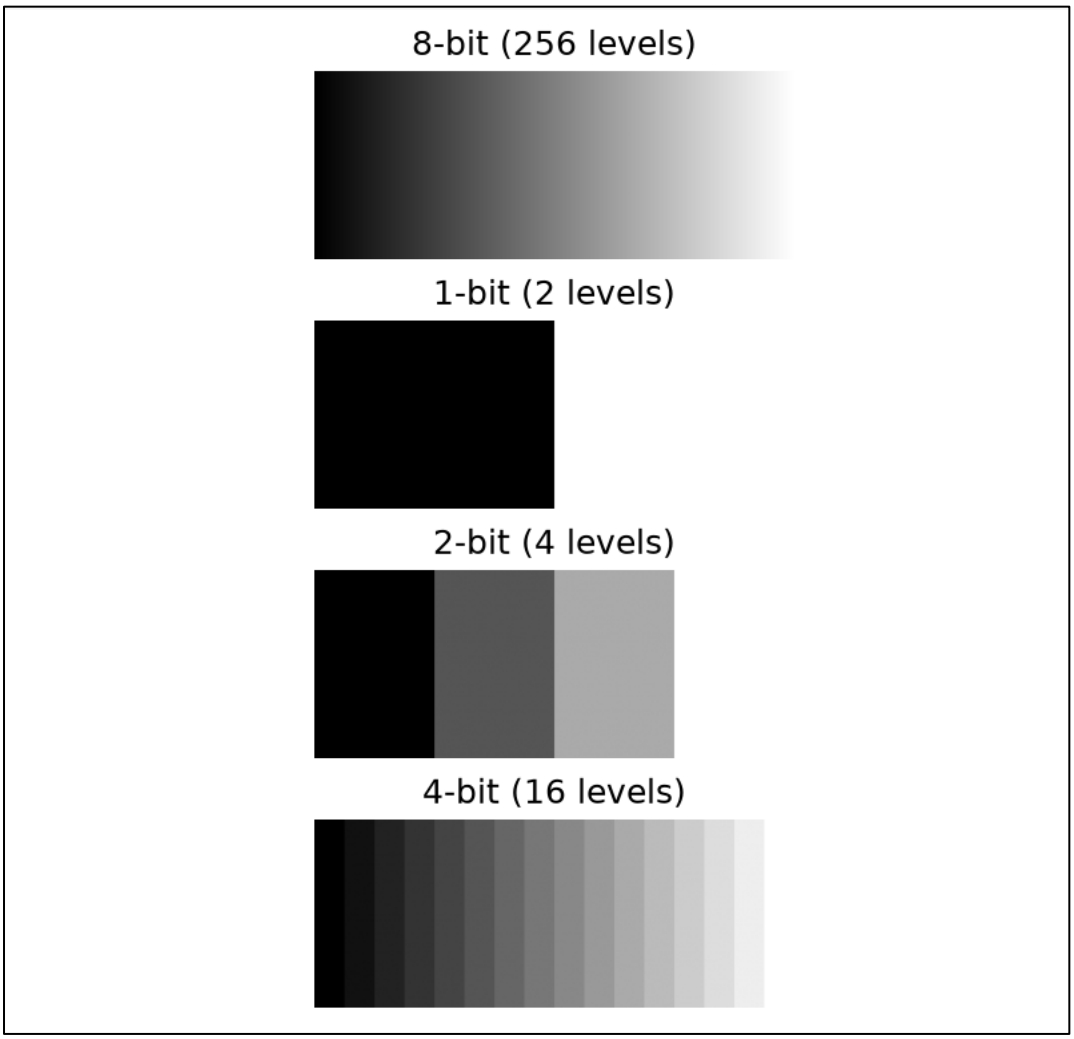
fig, axes = plt.subplots(4, 1, figsize=(8, 6))
for ax, (img, title) in zip(axes, grads):
    ax.imshow(img, cmap="gray", vmin=0, vmax=255)
    ax.set_title(title)
    ax.axis("off")
plt.tight_layout()
plt.savefig("bit_depth_figure.png", dpi=120)

print("\nStep 7: Bit depth ")
for img, title in grads:
    print(f"{title}: unique values = {len(np.unique(img))}")

plt.show()
```

## Output:

```
Step 7: Bit depth
8-bit (256 levels): unique values = 256
1-bit (2 levels): unique values = 2
2-bit (4 levels): unique values = 4
4-bit (16 levels): unique values = 16
```




### Step 8: Complete the Project

```
(.venv) PS E:\University\5th Semester\Digital Image Processing_Lab\Task Done\dip-lab-02> deactivate
PS E:\University\5th Semester\Digital Image Processing_Lab\Task Done\dip-lab-02> where python
PS E:\University\5th Semester\Digital Image Processing_Lab\Task Done\dip-lab-02>
```